CEX

Universal Typed Dictionary for LLM Agents

99 Kinds • 12 Pillars • 8F Pipeline • 7 Nuclei

Technical Whitepaper v1.0.0
2026-03-30

github.com/GatoaoCubo/cex

Contents

1. **The Problem**
2. **CEX in One Page**
3. **The 99 Kinds**
4. **The 12 Pillars**
5. **The 8F Pipeline**
6. **Builders (13 ISOs)**
7. **The 7 Nuclei**
8. **Taxonomy & Naming**

9. **Quality System**
10. **Multi-LLM Architecture**
11. **The Strange Loop**
12. **Quick Start**

1. The Problem

When you ask an LLM to build an agent, each model generates a different structure. Fields are invented ad-hoc. Formats are incompatible. No shared vocabulary, no schema, no quality contract.

> *CEX defines a **universal typed dictionary**: 99 artifact types (kinds), each with a schema, a builder, and quality gates. Any LLM produces structurally compatible artifacts.*

CEX is NOT	CEX IS
An agent framework (LangChain, CrewAI)	A construction dictionary any framework consumes
A prompt library	A typed taxonomy (prompts = 1 of 99 kinds)
Tied to one LLM	Multi-LLM (Claude, GPT, Gemini, Ollama)
A runtime engine	A build-time system producing validated artifacts

2. CEX in One Page

```
L1: DICTIONARY — 99 KINDS in 12 PILLARS
    Each kind: schema + builder (13 ISOs) + KC + examples
    Built via 8F PIPELINE (8 sequential functions)

L2: NUCLEI — 7 composed units (N01-N07)
    Each uses 12-25 kinds from L1
    N03 = Builder    N07 = Orchestrator

L3: INSTANCES — concrete project fills
    _instances/your-project/N01-N07/
```

Numbers: 99 kinds, 12 pillars, 8 functions, 7 nuclei, 1,287 builder ISOs, 99 KCs, 17 tools.

3. The 99 Kinds

A **kind** is a named artifact type with: definition, boundary (what it is NOT), pillar, max size, naming convention, and 8F function mapping. Source of truth: `.cex/kinds_meta.json`.

Pillar	Name	#	8F Fn	Example Kinds
P01	Knowledge	8	INJECT	knowledge_card, few_shot_example, glossary
P02	Model	11	BECOME	agent, axiom, mental_model, router
P03	Prompt	7	INJECT	system_prompt, prompt_template, chain
P04	Tools	18	CALL	function_def, mcp_server, cli_tool
P05	Output	4	GOVERN	response_format, formatter, parser
P06	Schema	6	CONSTRAIN	input_schema, interface, type_def
P07	Evals	10	GOVERN	benchmark, scoring_rubric, quality_gate
P08	Architecture	8	INJECT	agent_card, pattern, diagram
P09	Config	7	GOVERN	env_config, boot_config, feature_flag
P10	Memory	6	INJECT	learning_record, entity_memory
P11	Feedback	6	CONSTRAIN	quality_gate, guardrail, bugloop
P12	Orchestration	8	PRODUCE	workflow, dag, dispatch_rule, handoff

Boundary Discipline

Every kind declares what it is NOT: `agent` = "NOT a skill, NOT a tool, NOT a prompt". `workflow` = "NOT a chain, NOT a DAG". This prevents scope creep.

4. The 12 Pillars

Pillars group related kinds and map to 8F functions:

```
P01 Knowledge -----[INJECT]------> F3 (load knowledge)
P02 Model ---------[BECOME]------> F2 (load identity)
P03 Prompt --------[INJECT]------> F3 (load prompts)
P04 Tools ---------[CALL]--------> F5 (load tools)
P05 Output --------[GOVERN]------> F7 (validate output)
P06 Schema --------[CONSTRAIN]---> F1 (load constraints)
P07 Evals ---------[GOVERN]------> F7 (validate quality)
P08 Architecture --[INJECT]------> F3 (load patterns)
P09 Config --------[GOVERN]------> F1 (load config)
P10 Memory --------[INJECT]------> F3 (load memory)
P11 Feedback ------[CONSTRAIN]---> F7 (load guardrails)
P12 Orchestration -[PRODUCE]-----> F6/F8 (build flows)
```

Axiom AX03: One kind, one pillar. If a knowledge_card starts explaining a workflow, STOP. Separate artifacts.

5. The 8F Pipeline

Every artifact is produced by 8 sequential functions:

#	Function	Verb	Input	Output	Pillars
F1	CONSTRAIN	Restrict	Kind name	Schema + config	P06, P09
F2	BECOME	Be	Kind name	Builder identity (13 ISOs)	P02
F3	INJECT	Know	Kind + context	KCs + examples + memory	P01,P03,P08,P10
F4	REASON	Think	F1+F2+F3	Construction plan	LLM
F5	CALL	Do	Tools list	Tool manifest	P04
F6	PRODUCE	Generate	Plan + tools	Artifact draft	P12
F7	GOVERN	Evaluate	Draft + schema	Pass/Fail + score	P05,P07,P11
F8	COLLABORATE	Coordinate	Validated artifact	Saved+compiled+indexed	P12

F6→F7 Retry Loop

```
F6 PRODUCE -----> F7 GOVERN
     ^                 |
     |   PASS ---------+-------> F8 COLLABORATE (save)
     |                 |
     +-- FAIL ---------+  (max 2 retries, then REJECT)
```

Example

```
Intent: "quality gate for API responses"
[MOTOR] resolves: kind=quality_gate, pillar=P07
[RUNNER] F1: Load schema  F2: Load builder ISOs
         F3: Load KC      F4: Plan approach
         F5: Scan similar F6: Generate artifact
         F7: Validate     F8: Save + compile + signal
Output: quality_gate_api_response.md (score: 8.7)
```

6. Builders (13 ISOs)

Each kind has a builder: 13 ISO files in `archetypes/builders/{kind}-builder/`

#	ISO File	Purpose
1	bld_manifest	Capabilities, scope, version
2	bld_instruction	Step-by-step build instructions
3	bld_examples	Golden examples (input→output)
4	bld_schema	Required fields, types, constraints
5	bld_output_template	Expected output structure
6	bld_quality_gate	Kind-specific validation
7	bld_knowledge_card	Domain knowledge
8	bld_tools	Available tools
9	bld_system_prompt	LLM identity when building
10	bld_memory	Learning from past builds
11	bld_collaboration	Crew compositions
12	bld_architecture	Design patterns
13	bld_config	Build-time configuration

Total: 99 kinds × 13 ISOs = **1,287 instruction files**

> *Builders are the DNA of CEX. When an LLM loads a builder, it acquires specialized knowledge to construct that kind — regardless of which LLM executes.*

7. The 7 Nuclei

A nucleus composes 12–25 kinds into a domain-specific operational unit:

N	Domain	Kinds	CLI	Model	Role
N01	Intelligence	~21	gemini	2.5-pro (1M)	Research, analysis
N02	Marketing	~18	claude	sonnet	Copy, creative
N03	Engineering	25	claude	opus	The builder — builds all
N04	Knowledge	~23	gemini	2.5-pro (1M)	Indexing, RAG
N05	Operations	~23	codex	GPT-5.4	Code review, test
N06	Commercial	~18	claude	sonnet	Pricing, sales
N07	Admin	~23	pi+claude	opus-4-6	Orchestrator only

12 Core Kinds (All Nuclei Share)

Kind	Pillar	Purpose
agent	P02	Who the nucleus IS
system_prompt	P03	LLM identity
mental_model	P02	Decision trees
axiom	P02	Immutable truths
knowledge_card	P01	Domain knowledge
dispatch_rule	P12	Task routing
workflow	P12	Execution flow
quality_gate	P11	Validation
scoring_rubric	P07	Quality scoring
agent_card	P08	External interface
handoff	P12	Inter-nucleus comms
signal	P12	Completion notification

N03 Structure (34 Artifacts, 12 Subdirs)

```
N03_engineering/
  agents/        agent, axiom, mental_model
  prompts/       system_prompt, prompt_template, chain
  tools/         function_def
  config/        boot_config
  orchestration/ workflow, dispatch_rule, spawn_config, handoff, signal, dag
  knowledge/     knowledge_card, few_shot_example
  quality/       scoring_rubric, benchmark
  feedback/      quality_gate, guardrail
  architecture/  agent_card, pattern (x4)
  output/        response_format
  memory/        learning_record
  schemas/       input_schema, interface
```

8. Taxonomy & Naming

Artifact ID

Pattern: `{pillar}_{kind}_{topic}`. Examples: `p02_agent_security_reviewer`, `p01_kc_construction_laws`

Filename Conventions

Kind	Pattern	Example
agent	p02_agent_{name}.md	p02_agent_builder_nucleus.md
knowledge_card	p01_kc_{topic}.md	p01_kc_construction_laws.md
workflow	p12_wf_{name}.md	p12_wf_8f_pipeline.md
dispatch_rule	p12_dr_{scope}.yaml	p12_dr_nucleus_routing.yaml
scoring_rubric	p07_sr_{framework}.md	p07_sr_5d_quality.md

Builder ISO Naming

`bld_{function}_{kind}.md` — 13 functions: manifest, instruction, examples, schema, output_template, quality_gate, knowledge_card, tools, system_prompt, memory, collaboration, architecture, config.

Frontmatter Contract

Field	Type	Req	Description
id	string	YES	Unique (pillar_kind_topic)
kind	string	YES	One of 99 kinds
pillar	string	YES	P01–P12
title	string	YES	Human-readable
version	semver	YES	Semantic version
quality	float	YES	5D score (1-10)

tags	list	YES	Discovery tags
tldr	string	YES	One-line summary
density_score	float	REC	Signal-to-noise (0-1)

Open Variables

`{{mustache}}` syntax is deliberate. Open variables any LLM fills contextually. Core design feature, not bugs.

9. Quality System

Layer 1: H01–H07 Gates

Gate	Check	Severity
H01	Frontmatter integrity	HARD FAIL
H02	Kind matches request	HARD FAIL
H03	Naming convention	WARN
H04	References resolve	WARN
H05	Density ≥ 0.80	SOFT FAIL
H06	Size within max_bytes	SOFT FAIL
H07	Schema compliance	HARD FAIL

Layer 2: 12LP Checklist

12 checkpoints: frontmatter, kind match, pillar alignment, schema, density, completeness, uniqueness, references, boundary, size, naming, "stranger test."

Layer 3: 5D Scoring

Dimension	Weight	Measures
D1: Structural Integrity	25%	Frontmatter, schema, naming
D2: Content Density	20%	Signal-to-noise ratio
D3: Actionability	25%	Usable by stranger in 2min?
D4: Boundary Discipline	15%	Single kind, single responsibility
D5: Composability	15%	Clean interfaces, crew-compatible

Final = (D1×0.25)+(D2×0.20)+(D3×0.25)+(D4×0.15)+(D5×0.15)

Score	Tier	Action
≥ 9.5	GOLDEN	Reference examples

8.0-9.4	PUBLISH	Committed
7.0-7.9	ITERATE	Retry (max 2)
< 7.0	REJECT	Delete, restart

10. Multi-LLM Architecture

Nucleus	CLI	Model	Context	Cost	Strength
N07	pi+claude	opus-4-6	200K	$$$	Meta-reasoning
N03	claude	opus	200K	$$$	Construction
N05	codex	GPT-5.4	192K	$$	Code review
N04	gemini	2.5-pro	1M	$0	Knowledge
N01	gemini	2.5-pro	1M	$0	Research
N02	claude	sonnet	200K	$	Creative
N06	claude	sonnet	200K	$	Commercial

Fallback Chain

```
opus ($$$) → sonnet ($) → gemini-free ($0) → ollama-local ($0)
```

Zero-subscription users use Ollama. All Python tools work without API keys.

11. The Strange Loop

N03 was built using the 8F pipeline it defines. Self-referential: the factory IS the product.

3-Phase Bootstrap

Phase	Name	What
1	GENESIS	7 core artifacts written manually
2	AUTOPOIESIS	N03 builds remaining 19 via 8F
3	REWRITE	N03 rewrites originals with full context

> *Intentional Strange Loop (Hofstadter, 1979). Loop closes when Phase 3 outputs are strictly better than Phase 1 originals.*

10 Axioms

AX	Statement
01	Every artifact has YAML frontmatter. No exceptions.
02	Every kind has exactly one builder with 13 ISOs.
03	One kind, one pillar. Boundary is law.
04	The 8F pipeline is the only construction method.
05	Builders are instructions, not executors. Any LLM runs them.
06	Quality floor is 8.0. Below = not published.
07	Template-First: search before creating.
08	Compilation (.md → .yaml) is mandatory post-save.
09	The index is discovery. Re-index after every save.
10	Open variables ({{mustache}}) are deliberate.

12. Quick Start

```
git clone https://github.com/GatoaoCubo/cex.git
cd cex && pip install pyyaml
```

Boot

```
cex              # N07 orchestrator (pi + opus)
cex doctor       # Health check
cex status       # Monitor builders
cex stop         # Kill all
```

Build an Artifact

```
python _tools/cex_8f_motor.py --intent "quality gate for API"
python _tools/cex_8f_runner.py --kind quality_gate
python _tools/cex_doctor.py
python _tools/cex_compile.py P07_evals/quality_gate_api.md
```

Parallel Grid

```
powershell -File _spawn/spawn_solo.ps1 -nucleus n03 -task "build" -interactive
powershell -File _spawn/spawn_grid.ps1 -mission WAVE1 -interactive
powershell -File _spawn/spawn_monitor.ps1
```

CEX v1.0.0 • 99 kinds • 12 pillars • 8F pipeline • 7 nuclei • 1,287 builder ISOs
github.com/GatoaoCubo/cex